RESUMEN EJECUTIVO · MANAGEMENT

RoslynGate

Menos contexto, más precisión: cómo hacer que un agente de IA entienda una base .NET real sin leerla entera.

Una capa semántica entre los agentes de IA y las soluciones .NET. Diseñada para equipos con repositorios grandes, proyectos legacy y código que no perdona una referencia inventada.

01 — EL PROBLEMA

El agente ve texto. El negocio necesita certeza.

Cuando un agente busca símbolos con grep, encuentra coincidencias. Cuando el código tiene overloads, partial classes, interfaces, código generado y proyectos legacy, una coincidencia no es una respuesta.

1 Contexto caro

Leer archivos completos para encontrar una referencia consume tokens, tiempo y atención del modelo.

2 Respuestas frágiles

Un falso positivo puede llevar a editar el archivo equivocado. Un falso negativo puede ocultar impacto real.

3 Soluciones grandes

Noventa y nueve proyectos y paquetes legacy no son un caso de laboratorio; son el día a día.

4 Herramientas genéricas

grep es excelente para texto. No entiende símbolos. Un IDE entiende símbolos, pero no alimenta a un agente por MCP.

> *La oportunidad no es “otro buscador”. Es darle al agente la respuesta semántica mínima para que no tenga que leer medio repositorio.*

02 — LA PROPUESTA

Una capa semántica para agentes .NET

RoslynGate carga la solución con MSBuild y responde preguntas con Roslyn: definición, referencias, jerarquía y búsqueda por nombre. El agente recibe JSON compacto con archivo y línea.

Precisión

Resuelve símbolos, no coincidencias de texto. Distingue overloads, tipos y proyectos.

Economía

Devuelve lo mínimo: ubicación, tipo de uso y paginación accionable.

Integración

Habla MCP. Claude Code y otros agentes lo usan sin integraciones a medida.

Qué obtiene el equipo

ANTES	CON ROSLYNGATE
“Busca ‘Doors’ y mira archivos”	“Resuelve el símbolo y devuelve sus referencias semánticas”.
El agente lee archivos completos	El agente recibe `file:line` y solo lee si hace falta.
Overloads resueltos por prueba y error	Candidatos o tipos de parámetros.
Resultados que dependen del prompt	Orden estable, paginación y errores accionables.

> El binario es self-contained: el runtime va incluido. El SDK .NET sigue siendo necesario porque MSBuild lo usa para leer los proyectos. RoslynGate respeta el SDK que fija `global.json`.

03 — EVIDENCIA

El ahorro se midió, no se estimó

Dos pruebas A/B con Claude Code sobre una solución legacy de 99 proyectos. Mismo prompt, mismo modelo, con y sin RoslynGate.

33,9%
menos tokens · 3 símbolos

21,2%
menos tokens · 6 símbolos

30%
menos coste · 3 símbolos

22%
menos coste · 6 símbolos

CONSULTA	TOKENS CON	TOKENS SIN	DURACIÓN CON	DURACIÓN SIN
3 símbolos	94.158	142.506	88,1 s	29,2 s
6 símbolos	103.904	131.865	102,3 s	61,8 s

La lectura correcta

- El ahorro es real en tokens y coste, no solo en caracteres.
- La latencia empeora porque cada proceso nuevo carga MSBuild; en una sesión interactiva el workspace se conserva.
- En seis símbolos RoslynGate devolvió más referencias que el baseline: no fue truncamiento.
- Es una muestra; el porcentaje depende de la consulta y del repositorio.

04 — DIFERENCIACIÓN

No compite con grep. Compite con la ambigüedad.

grep seguirá siendo la herramienta correcta para texto libre. RoslynGate cubre la pregunta que grep no puede responder: “¿qué símbolo es este y quién lo usa de verdad?”.

HERRAMIENTA	FORTALEZA	LÍMITE PARA UN AGENTE
grep / rg	Rápido, simple, barato.	Texto, no semántica.
IDE / LSP	Excelente para una persona.	No es un contrato MCP para agentes.
Analyzers	Reglas de calidad.	No responde consultas ad hoc.
Índices globales	Cobertura amplia.	No usan la compilación real del repo.
RoslynGate	Semántica .NET, compacta, read-only.	Necesita SDK y snapshot fresco.

> *La diferenciación no es “más rápido que grep”. Es “la respuesta correcta para un símbolo, en el formato que un agente puede consumir”.*

Por qué ahora

Los agentes ya escriben y editan código. La limitación dejó de ser generar texto y pasó a ser entender el repositorio con precisión. Ese es el espacio de RoslynGate.

05 — ADOPCIÓN

Cómo entra en un equipo .NET

La adopción es incremental. No requiere cambiar el compilador ni el flujo de build; requiere instalar el paquete, tener el SDK y configurar MCP.

1 Instalar

`npm` o `bun`. El wrapper descarga el binario self-contained y verifica el checksum.

2 Configurar

`roslyngate setup claude --solution . --skill` crea `.mcp.json` y una skill de proyecto.

3 Medir

Un A/B por prompt representativo con `claude-ab.ps1`. No se promete un número sin medirlo.

4 Escalar

Watcher incremental, recarga explícita, filtros por proyecto y análisis de impacto.

Riesgos y mitigación

RIESGO	MITIGACIÓN
SDK ausente o distinto	Preflight, respeto a `global.json` y mensaje accionable.
Workspace viejo tras editar	Watcher incremental, `stale` visible y `reload_workspace`.
Latencia inicial	Prewarm, sesión persistente y benchmark separado.
Expectativa exagerada	Publicar resultados con muestra, coste y límites.

06 — HORIZONTE

De herramienta a plataforma

La base ya está separada en capas y expone un contrato. Eso permite crecer hacia casos de mayor valor sin reescribir el motor.

Frescura en vivo

Editar y consultar sin reiniciar; recarga explícita solo para cambios estructurales.

Precisión de overloads

Resolver por tipos de parámetros y reducir vueltas del agente.

Impacto de cambios

Qué implementaciones y llamadas se rompen antes de tocar una interfaz.

CI y revisión

Verificación semántica en pipelines y PRs, no solo en el chat.

Arquitectura

Mapas de dependencias reales entre proyectos y capas.

Equipos

Skill compartida, setup reproducible y métricas por prompt.

> *El objetivo no es que el agente lea menos por leer menos. Es que lea lo correcto.*

READ-ONLY · SEMÁNTICO · MCP · MEDIBLE · ESCALABLE